DiceMind — Mestre da Probabilidade

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	DiceMind — Mestre da Probabilidade
Gerente de Projeto	Higor de Medeiros Santana
Gerente/Responsável de GCS-DevOps	Higor de Medeiros Santana
Fábrica de Software / Squad	Equipe DiceMind — PAC (Projeto de Aprendizagem Colaborativa)
Repositório principal	https://github.com/Higor-de-Medeiros-Santana/PAC-JOGO

Histórico de Revisões

Data	Versão	Descrição	Autor
20/09/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD) para o projeto DiceMind.	Higor de Medeiros Santana

Sumário

1. Introdução 3
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 4
1.5 Visão Geral 4
2. Cultura DevOps e Papéis na Gerência de Configuração 4
3. Ferramentas, Ambientes e Infraestrutura 5
3.1 Ferramentas do Pipeline 5
3.2 Ambientes e Infraestrutura 6
3.3 Estratégia de Branching e Versionamento 7
4. Identificação da Configuração 7
4.1 Itens de Configuração 7
4.2 Convenção de Nomenclatura 7
4.3 Estrutura do Repositório 8
5. Integração Contínua (CI) 8
5.1 Etapas do Pipeline de CI 8
5.2 Política de Pull Request e Revisão de Código 9
5.3 Critérios de Qualidade (Quality Gate) 9
6. Entrega e Implantação Contínua (CD) 9
6.1 Etapas do Pipeline de CD 9
6.2 Estratégias de Implantação 9
6.3 Rollback e Recuperação 10
7. Infraestrutura como Código (IaC) 10
7.1 Versionamento de Infraestrutura 10
7.2 Ambientes Efêmeros 10
8. Controle de Configuração e Mudança 11
8.1 Processo de Solicitação de Mudança 11
8.2 Comitê de Controle de Mudança (CCM) 11
9. Baselines e Releases 11
9.1 Geração Automática de Baseline 11
9.2 Notas de Versão (Release Notes) 11
10. Estimativa do Status de Configuração 11
10.1 Monitoramento e Observabilidade 11
10.2 Métricas DORA 12
10.3 Relatórios e Auditorias 12
11. Marcos 12
12. Treinamento e Recursos 12
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 13
14. Quadro-Resumo do Pipeline CI/CD 13

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe do DiceMind, garantindo rastreabilidade, versionamento e controle de mudanças do jogo (código-fonte, cenas, assets, builds e documentação), apoiado por pipelines automatizados de build, teste e publicação.

1.2 Escopo

Este PGC é destinado a todos os integrantes da equipe DiceMind, responsável pelo desenvolvimento do jogo DiceMind — Mestre da Probabilidade, um jogo educacional sobre probabilidade, de temática de dados/cassino, desenvolvido na Godot Engine com a linguagem GDScript e distribuído em builds para Windows e HTML5/WebAssembly. O projeto é desenvolvido no contexto do Projeto de Aprendizagem Colaborativa (PAC) e documentado também nas disciplinas de Gerenciamento de Projetos e de DevOps.

Estão no escopo deste plano: o código-fonte (GDScript), as cenas e os recursos da Godot, os assets (imagens, áudio e fontes), as configurações de projeto e de exportação, os workflows de automação, os builds gerados, a documentação técnica e os documentos de gestão do projeto (Termo de Abertura, EAP e Plano de Tratamento de Riscos). O jogo é distribuído como aplicação autônoma (desktop e web), sem servidores próprios a provisionar.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

Termo	Significado
PAC	Projeto de Aprendizagem Colaborativa, contexto acadêmico em que o DiceMind é desenvolvido
EAP	Estrutura Analítica do Projeto (Work Breakdown Structure)
Godot Engine	Motor de jogos de código aberto utilizado no desenvolvimento do DiceMind
GDScript	Linguagem de script da Godot Engine, usada na lógica do jogo
Export Preset	Configuração de exportação da Godot (plataforma, opções e destino), registrada em export_presets.cfg
WebAssembly (WASM)	Formato que permite executar o build web (HTML5) do jogo no navegador
GUT	Godot Unit Test, framework de testes automatizados para GDScript

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Termo de Abertura do Projeto DiceMind, v1.1.
- Estrutura Analítica do Projeto (EAP) e Dicionário da EAP do DiceMind, v1.1.
- Plano de Tratamento de Riscos do DiceMind, v1.0.
- Pacote de entrega do PAC (DiceMind-pac-3_1), incluindo o manual de reconstrução do projeto.
- Documentação oficial da Godot Engine (exportação de projetos e uso por linha de comando) e do GitHub Actions.

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

Situação atual e evolução planejada: hoje o build do DiceMind é gerado manualmente, por meio da exportação pelo editor da Godot, e o repositório PAC-JOGO no GitHub concentra os artefatos do projeto. Este plano define o padrão de GCS a seguir e o pipeline-alvo de CI/CD (GitHub Actions), que será implantado de forma incremental, conforme os marcos da Seção 11. Enquanto a automação não estiver ativa, as verificações correspondentes são realizadas manualmente, por meio de revisão em Pull Request e de checklist de release.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics). **No DiceMind:** Higor de Medeiros Santana (Gerente de Projeto), com apoio de Gustavo Henrique Forte (Representante do Negócio) na priorização do backlog. Carlos Augusto Xavier Lourenço Junior (Patrocinador) é consultado em mudanças que afetem escopo ou prazo.
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC. **No DiceMind:** Higor de Medeiros Santana, Responsável GCS-DevOps: mantém os workflows do GitHub Actions, as regras de proteção da branch main, os export presets e as releases.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado. **No DiceMind:** Jenifer Alves Scherpinski (Líder Técnica) e Felipe Silveira (Desenvolvedor). A Líder Técnica é a revisora preferencial de Pull Requests com impacto estrutural.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).* **No DiceMind:** Por não haver servidores em produção, o papel se resume ao acompanhamento das releases publicadas (disponibilidade dos downloads e da versão web) e dos bugs reportados, a cargo do Responsável GCS-DevOps; o rollback consiste em republicar a versão anterior (ver 6.3).
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD. **No DiceMind:** Jenifer Alves Scherpinski e Felipe Silveira mantêm os testes automatizados (GUT) das regras do jogo; Gustavo Henrique Forte conduz a validação de aceitação (playtest) em homologação.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline. **No DiceMind:** O Responsável GCS-DevOps ativa o Dependabot para as Actions utilizadas e garante que nenhuma credencial seja versionada (verificação feita na revisão de cada PR).
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline. **No DiceMind:** Composto por Higor de Medeiros Santana, Jenifer Alves Scherpinski e Gustavo Henrique Forte; Carlos Augusto Xavier Lourenço Junior (Patrocinador) é consultado quando houver impacto em escopo, prazo ou custo.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas adotadas e planejadas no projeto DiceMind. Situação atual: o build é exportado manualmente pelo editor da Godot; as ferramentas de automação marcadas como (planejado) serão implantadas conforme os marcos da Seção 11.

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub (repositório PAC-JOGO)	Versionamento de código-fonte, cenas, assets, configurações, workflows e documentação
Orquestração de CI/CD	GitHub Actions (planejado)	Automação de lint, testes, exportação dos builds e publicação
Motor e empacotamento	Godot Engine (versão fixada no projeto) e export presets Windows Desktop e Web	Geração dos builds versionados; exportação em modo headless no pipeline
Registro de artefatos	GitHub Releases e artefatos de workflow	Armazenamento e versionamento dos builds Windows e HTML5 por versão (tag)
Testes e análise estática	GUT (Godot Unit Test) e gdtoolkit — gdlint/gdformat (planejado)	Testes das regras do jogo e verificação de padrões de codificação GDScript
Infraestrutura como Código	Workflows YAML, export_presets.cfg e dependabot.yml versionados	Configuração de build e publicação versionada (não há servidores a provisionar)
Orquestração de containers	Não aplicável	O DiceMind é distribuído como aplicação desktop e web estática; os pipelines rodam em runners do GitHub Actions
Gestão de segredos	GitHub Actions Secrets	Credenciais e tokens usados nos pipelines, nunca versionados no repositório
Monitoramento e observabilidade	Histórico do GitHub Actions, alertas de falha de workflow e Issues de bugs	Acompanhamento da saúde dos pipelines e dos problemas reportados
Gestão de dependências	Dependabot (atualização das GitHub Actions utilizadas)	Atualização controlada das dependências dos pipelines, submetida ao fluxo de PR
Rastreamento de tarefas	GitHub Issues (vinculadas às entregas da EAP)	Vinculação de commits/PRs a itens de trabalho

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Máquinas locais dos desenvolvedores (editor da Godot) e execução do CI em runners efêmeros do GitHub Actions a cada Pull Request.	Abertura/atualização de Pull Request
Homologação / Staging	Pré-release com os builds Windows e Web, para playtest da equipe e validação de aceitação pelo Representante do Negócio.	Merge na branch principal (main) com CI aprovado
Produção	Release oficial publicada no GitHub Releases (build Windows) e versão Web (HTML5) publicada para acesso via navegador, para uso final e avaliação.	Aprovação do gate de release e criação da tag vX.Y.Z

Não há DNS, endereços IP, balanceadores ou bancos de dados próprios a gerenciar: o DiceMind é distribuído como aplicação desktop (Windows) e como build web estática (HTML5/WebAssembly). Eventuais credenciais (por exemplo, tokens de publicação) são armazenadas em GitHub Actions Secrets, nunca no repositório.

3.3 Estratégia de Branching e Versionamento

Adota-se o GitHub Flow, uma variação de Trunk-Based Development adequada a equipes pequenas, com branches de vida curta, para viabilizar a integração contínua.

- main — sempre em estado jogável (o projeto abre no editor da Godot e a cena principal executa); protegida por regras de PR obrigatório e, a partir do Marco M1, por pipeline verde.
- feature/<issue>-<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança; correções de bugs seguem o mesmo padrão com o prefixo fix/.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes da publicação (por exemplo, congelamento para uma entrega do PAC).
- hotfix/<descrição> — correções emergenciais aplicadas sobre a versão publicada e retro-integradas à main.
- Cenas e recursos da Godot (.tscn/.tres) são arquivos de texto, mas conflitos de merge nesses arquivos são frequentes: duas pessoas não devem editar a mesma cena ao mesmo tempo. A edição de cenas compartilhadas é combinada na Issue/PR, e as mudanças são mantidas pequenas e integradas com frequência.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor...), que permitem classificar as mudanças e gerar o changelog e as notas de versão. A versão do jogo é registrada em application/config/version do project.godot e deve coincidir com a tag de release.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte GDScript do jogo e dos testes automatizados (GUT)
- Cenas (.tscn), recursos (.tres) e configurações do projeto (project.godot)
- Assets do jogo: imagens, áudio, fontes e demais recursos, com origem e licença registradas
- Definições de pipeline como código (.github/workflows) e configuração do Dependabot (.github/dependabot.yml)
- Configurações de exportação (export_presets.cfg) para Windows e Web — exceto credenciais e chaves de assinatura, que devem ficar em ambientes seguros (GitHub Actions Secrets).
- Builds gerados (executável Windows e pacote Web) — itens de configuração derivados: não são versionados no repositório, e sim publicados nas Releases e identificados pela tag correspondente
- Scripts de migração de banco de dados (N/A)
- Documentação técnica e de gestão: README, manual de reconstrução do projeto, Termo de Abertura, EAP, Plano de Tratamento de Riscos, este PGC e ADRs (Architecture Decision Records)

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<issue>-<descrição-curta>	feature/23-tela-de-resultados

Item	Convenção	Exemplo
Branch de correção	fix/<issue>-<descrição-curta>; hotfix/<descrição> para correções emergenciais	fix/31-calculo-probabilidade
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(dados): adiciona rolagem animada de dados
Tag de release	Versionamento Semântico vX.Y.Z	v1.2.0
Artefato de build	DiceMind-<plataforma>-v<versão>.zip	DiceMind-windows-v1.2.0.zip
Arquivos Godot (cenas, scripts, assets)	snake_case, conforme o guia de estilo da Godot	tela_resultado.tscn, dado_controller.gd

4.3 Estrutura do Repositório

Este plano define a seguinte organização para o repositório PAC-JOGO (raiz do projeto Godot):

- **project.godot e export_presets.cfg** — configuração do projeto e dos presets de exportação (Windows e Web).
- **scenes/** — cenas da Godot (.tscn), organizadas por funcionalidade.
- **scripts/** — scripts GDScript (.gd) do jogo.
- **assets/** — imagens, áudio e fontes, com arquivo de créditos e licenças.
- **addons/** — plugins de terceiros (por exemplo, GUT), com versão registrada.
- **tests/** — testes automatizados (GUT).
- **docs/** — documentação técnica e de gestão (PGC, Termo de Abertura, EAP, Plano de Riscos, manual de reconstrução e relatórios).
- **.github/** — workflows do GitHub Actions e configuração do Dependabot.
- **README.md e .gitignore** — instruções de uso e de build; exclusão do cache do editor (.godot/) e dos builds gerados.

Os builds gerados (pasta de exportação local) não são versionados; ficam nas Releases. O GitHub rejeita arquivos individuais acima de 100 MB; caso o projeto passe a ter assets desse porte, será avaliado o uso de Git LFS.

5. Integração Contínua (CI)

A partir do Marco M1 (Seção 11), todo Pull Request para a branch principal dispara automaticamente um pipeline de CI no GitHub Actions, e nenhuma mudança é integrada sem que ele seja concluído com sucesso. Até lá, o build é gerado manualmente no editor da Godot e a verificação é feita por revisão de PR e checklist manual.

5.1 Etapas do Pipeline de CI

1. Checkout do código e preparação da Godot Engine em modo headless, na versão fixada do projeto (com cache dos export templates).
2. Importação dos recursos e verificação de build: o projeto carrega sem erros.
3. Análise estática (gdlint) e verificação de formatação (gdformat) dos scripts GDScript.

4. Testes unitários (GUT) das regras de probabilidade, pontuação e lógica de jogo.
5. Testes de integração: carga das cenas principais e fluxo básico de uma partida, executados em modo headless.
6. Verificação de dependências e segurança: addons e Actions com versões fixadas e nenhuma credencial versionada.
7. Exportação dos builds (Windows e Web) e publicação como artefatos do workflow.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outro integrante da equipe antes do merge ("quatro olhos"); mudanças estruturais (arquitetura, cenas centrais, pipeline) são revisadas preferencialmente pela Líder Técnica.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate). Antes do Marco M1, o autor anexa ao PR o checklist manual: o projeto abre, a cena principal executa e uma partida básica funciona.

5.3 Critérios de Qualidade (Quality Gate)

Um PR só é integrado quando todos os critérios abaixo são atendidos:

- Zero erros bloqueantes de lint (gdlint) e scripts formatados (gdformat).
- Todos os testes automatizados aprovados; toda regra de probabilidade ou de pontuação criada ou alterada acompanha teste correspondente (sem meta percentual de cobertura, pois a GUT não a mede nativamente).
- Exportação dos builds Windows e Web concluída sem erros.
- Nenhuma credencial ou arquivo sensível versionado (verificado na revisão).
- Pelo menos 1 aprovação de revisão e nenhuma conversa de revisão pendente.

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

6.1 Etapas do Pipeline de CD

8. Publicação automática dos builds (Windows e Web) como pré-release no GitHub — ambiente de Homologação/Staging.
9. Testes de aceitação em staging: playtest conduzido pela equipe, seguindo roteiro de partida completa, com validação do Representante do Negócio (a automação E2E é evolução futura).
10. Gate de aprovação para produção: aprovação manual do Gerente de Projeto e da Líder Técnica.
11. Publicação em produção: promoção da pré-release a Release oficial (Windows) e publicação da versão Web, conforme a estratégia definida (ver 6.2).
12. Testes de fumaça (smoke tests) pós-publicação: baixar o artefato publicado, abrir o jogo e concluir uma partida; acessar a versão Web e confirmar o carregamento.
13. Geração automática de baseline/tag de release e notas de versão (changelog) no GitHub Releases.

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade. **No DiceMind:** Não se aplica: não há instâncias de servidor a substituir.
Blue-Green	Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato. **No DiceMind:** Parcialmente aplicável: a versão anterior permanece disponível nas Releases e a nova só é promovida após validação em homologação.
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança. **No DiceMind:** O equivalente é liberar a pré-release a um grupo restrito de playtesters antes da Release oficial.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases. **No DiceMind:** Aplicável: mecânicas ou modos de jogo em desenvolvimento ficam desativados por constante/configuração do projeto até estarem prontos, permitindo integrar cedo na main.

Estratégia adotada: publicação direta da Release após validação prévia em homologação, com feature flags para mecânicas ainda em desenvolvimento.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. As versões anteriores permanecem disponíveis nas Releases do GitHub (builds e tags), permitindo o rollback: a versão Web é republicada a partir da tag anterior e a release com problema é marcada como retirada. Falhas nos testes de fumaça impedem a promoção e disparam a correção por meio de uma branch hotfix/.

7. Infraestrutura como Código (IaC)

O DiceMind não possui infraestrutura de servidores a provisionar. O princípio de Infraestrutura como Código é aplicado à configuração de build e publicação: os workflows do GitHub Actions, os export presets da Godot e a configuração do Dependabot são descritos em arquivos versionados e submetidos ao mesmo fluxo de Pull Request e revisão usado para o código do jogo.

7.1 Versionamento de Infraestrutura

- Alterações nos workflows, nos export presets e na configuração de dependências seguem o mesmo processo de PR + revisão + execução do pipeline, que valida a alteração antes do merge.
- Não há state file de infraestrutura: o estado da configuração é o próprio commit/tag do repositório. As versões da Godot Engine, dos export templates e das Actions são fixadas em arquivos versionados, tornando os builds reproduzíveis a partir de qualquer tag e complementando o manual de reconstrução do projeto.

7.2 Ambientes Efêmeros

Cada execução do pipeline roda em um runner efêmero do GitHub Actions, criado limpo e descartado ao final, garantindo builds independentes da máquina de cada desenvolvedor. Os builds gerados em Pull Requests ficam disponíveis como artefatos temporários do workflow, para playtest da mudança, e expiram automaticamente conforme o período de retenção configurado.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho (Issue do GitHub, vinculada à entrega da EAP) → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança. Mudanças que alterem escopo, cronograma ou riscos do projeto exigem, além disso, a atualização dos artefatos de gestão correspondentes (Termo de Abertura, EAP, Plano de Tratamento de Riscos) e são tratadas conforme a Seção 8.2.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações de escopo ou da EAP, troca de versão principal da Godot Engine, mudança na estratégia de branching ou nos pipelines, inclusão de assets ou addons com licença restritiva, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

Composição do CCM: Higor de Medeiros Santana (Gerente de Projeto e Responsável GCS-DevOps), Jenifer Alves Scherpinski (Líder Técnica) e Gustavo Henrique Forte (Representante do Negócio). O Patrocinador, Carlos Augusto Xavier Lourenço Junior, é consultado quando houver impacto em escopo, prazo ou custo. A solicitação é feita por Issue, e a decisão é registrada na própria Issue.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada publicação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, aos builds publicados na Release e às versões da Godot e dos export presets utilizados. Entregas formais às disciplinas (por exemplo, o pacote do PAC) também são marcadas com a tag correspondente.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo GitHub Releases a partir dos Pull Requests e commits (Conventional Commits) desde a última tag, publicadas junto à release no repositório e complementadas por um resumo das mudanças visíveis ao jogador.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Como o DiceMind não possui servidores em produção, o monitoramento concentra-se em: histórico de execuções e alertas de falha do GitHub Actions, Issues de bugs reportados por playtesters e usuários, e verificação da disponibilidade dos downloads e da versão Web após cada release, permitindo identificar rapidamente falhas decorrentes de uma publicação.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção. **No DiceMind:** Número de Releases publicadas por período (tags no GitHub).
Lead Time for Changes	Tempo entre o commit e a implantação em produção. **No DiceMind:** Tempo entre o commit e a Release que o contém.
Change Failure Rate	Percentual de implantações que resultam em falha em produção. **No DiceMind:** Percentual de releases que exigiram hotfix ou rollback.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha. **No DiceMind:** Tempo entre a detecção do problema e a publicação da correção.

As métricas são coletadas a partir do histórico de tags, Releases e execuções do GitHub Actions e registradas ao fim de cada entrega.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos. Ao fim de cada entrega, o Responsável GCS-DevOps realiza uma verificação de configuração: itens de configuração versionados, tags conferidas com as releases, regras de proteção da main ativas e workflows funcionando.

11. Marcos

Os marcos abaixo são sequenciais; cada um é concluído quando o critério indicado é atendido e registrado no repositório.

Marco	Entrega	Critério de conclusão
M0	Repositório organizado	Estrutura de diretórios definida (4.3), README e .gitignore criados, PGC em docs/ e proteção da branch main ativa.
M1	CI implantado	Workflow de Pull Request executando lint, testes GUT e exportação dos builds Windows e Web.
M2	CD de homologação	Pré-release publicada automaticamente após o merge na main, com roteiro de playtest definido.
M3	Primeira release automatizada	Tag SemVer, Release com notas geradas, builds Windows e Web publicados e teste de fumaça executado.
M4	Dependências e métricas	Dependabot ativo para as Actions e primeira coleta das métricas DORA registrada.

12. Treinamento e Recursos

Os recursos necessários não geram custo adicional ao projeto, dentro dos limites do plano gratuito do GitHub.

- **Ferramentas:** Git e GitHub (repositório, Pull Requests, Issues, Actions e Releases), Godot Engine na versão fixada do projeto, GUT e gdtoolkit.

- **Pessoal:** o Responsável GCS-DevOps (Higor) configura e mantém os pipelines; Jenifer e Felipe mantêm os testes e revisam PRs; Gustavo conduz a validação de aceitação.
- **Treinamento:** sessão de nivelamento para toda a equipe sobre fluxo de branches e PRs, Conventional Commits, resolução de conflitos em cenas da Godot e leitura dos resultados do pipeline; material de apoio: documentação oficial da Godot e do GitHub Actions.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas, plugins e assets de terceiros integrados ao produto também são itens de configuração e devem ter versão e origem explícitas. Como a Godot Engine não possui gerenciador de pacotes com arquivo de dependências travado (lockfile), as versões da engine, dos addons e das Actions são fixadas em arquivos versionados.

- Toda nova dependência (addon, asset ou biblioteca) é revisada em Pull Request, com registro de origem e licença (por exemplo, arquivo de créditos em assets/).
- Atualizações das GitHub Actions utilizadas são propostas automaticamente pelo Dependabot e submetidas ao mesmo fluxo de PR.

O projeto não possui subcontratados nem fornecedores externos de software. As dependências de terceiros limitam-se à Godot Engine (licença MIT), aos addons utilizados (por exemplo, GUT) e a assets externos, cujas licenças são registradas no repositório.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Lint (gdlint/gdformat), testes GUT e exportação dos builds Windows e Web, publicados como artefatos do PR.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
CD — Staging	Publicação automática da pré-release (Windows e Web) e playtest de aceitação.	Merge na branch principal
Gate de Release	Aprovação manual do Gerente de Projeto e da Líder Técnica.	Sucesso do playtest em homologação
CD — Produção	Publicação da Release e da versão Web, testes de fumaça, geração da baseline/tag e das notas de versão.	Aprovação do gate de release

Nota: *este plano descreve o padrão de GCS e o pipeline-alvo do DiceMind. Na data desta versão o build ainda é exportado manualmente no editor da Godot; a automação será implantada de forma incremental conforme os marcos da Seção 11, e este documento deve ser revisado a cada marco concluído.*